purposes. You may have the goal simultaneously of ignoring the casing for uniqueness enforcement.

How can you accomplish both of those goals at once?

Significant Casing And Unique Constraints

One technique for email addresses and casing is to convert email addresses to lower case before they're stored. A downside of this technique is that it discards the original casing.

An alternative technique is to accept and store the input data in whatever casing the user has supplied at write time, but ignore the case information when the email address is presented at query time. How can this be accomplished?

The PostgreSQL module citext[12] provides a case insensitive character type. Using this character type as a column type allows you to achieve this write and read behavior.

To set this up, enable the citext extension in the Rideshare database. Enter this SQL statement into psql:

```
CREATE EXTENSION IF NOT EXISTS citext;
```

As you've done in earlier examples, enable the extension using a Migration. citext is not in Rideshare. Review extensions with \dx.

To test this out, create a new table in the Rideshare database but create it in a separate temp schema. Use the temp schema for tables that aren't part of the application schema. At the end you can drop the temp schema.

From psql, enter this statement to create Schema if it doesn't exist:

```
CREATE SCHEMA IF NOT EXISTS temp;
```

Generate a new migration and use the following example as the implementation. The Migration enables the citext extension and creates a customers table in the temp schema.

ruby/migration_add_customers_table.rb

```
class AddCustomersTable < ActiveRecord::Migration[7.0]
  def change
    enable_extension 'citext' unless extension_enabled?('citext')

    # Assumes a `temp` schema exists
    create_table 'temp.customers' do |t|
```

12. https://www.postgresql.org/docs/current/citext.html